

BACKEND ENGINEER · COMMERCE DOMAIN SPECIALIST

도전을 두려워하 지 않는 Chan Yeong 개발 기록

커머스 복지몰의 주문·배송·결제 도메인 전반을 담당하며, **MSA**
분산 트랜잭션과 비동기 메시징으로 장애에 강하고 변경에 유연한
시스템을 **AI-Driven Development Lifecycle** 관점으로 개발해
왔습니다.

4.1 yrs

Commerce
Backend

MSA

Saga · Distributed
Transaction

AI-DLC

AI-Driven
Development

**Test
Code**

Testcontainers ·
TDD



김찬영

BACKEND ENGINEER

COMPANY
DOMAIN
STACK
EDUSK엠앤서비스
Commerce
Spring · MSA
성결대학교

저는 커머스 도메인에 특화된 개발자로서, 커머스 복지몰에서의 복잡한 **주문 · 배송비 · 결제** 프로세스를 전담하였습니다. **MSA 환경에서의 분산 트랜잭션**을 제어하며 상품과 배송비의 주문 구조, 취소/교환/반품 등 다양한 정책을 반영하며 안정적인 운영에 기여해 왔습니다.

레거시한 절차지향 코드를 **객체지향 구조로 리팩토링**하고, 시스템 아키텍처를 개선하는 데 집중해 왔습니다. 변경에 강한 구조를 만들어 유지보수성과 협업 효율을 함께 향상시켰으며, XSS 대응과 ISMS 인증 작업을 수행해 보안 측면에서도 신뢰할 수 있는 서비스를 구축했습니다.

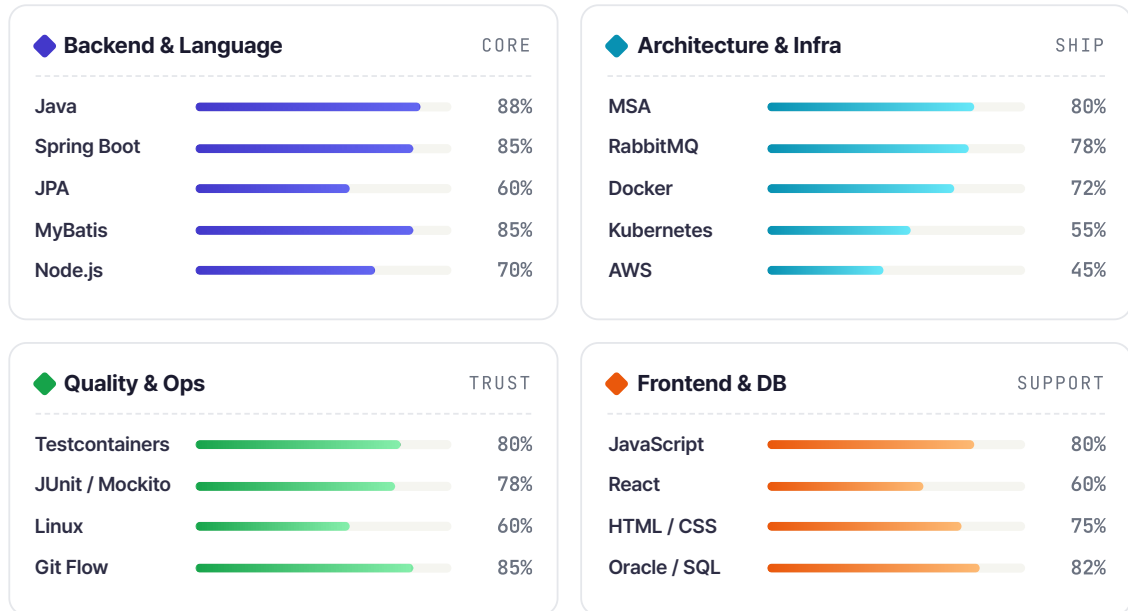
또한 **RabbitMQ**로 서비스 간 결합도를 낮추는 아키텍처를 설계하여 확장성과 장애 대응력을 강화했고, 반복 운영 업무를 자동화한 백오피스 기능으로 CS 처리 시간을 약 90% 이상 절감한 경험이 있습니다.

나아가 **Harness Engineering**을 통해 안전한 운영 시스템 접근 및 실무형 AI 개발 체계를 구축하여 AI 생산성을 크게 향상시켰고, **AI-DLC 개발 방법론**을 도입하여 개발 공수 기간을 대폭 감소시켰습니다.

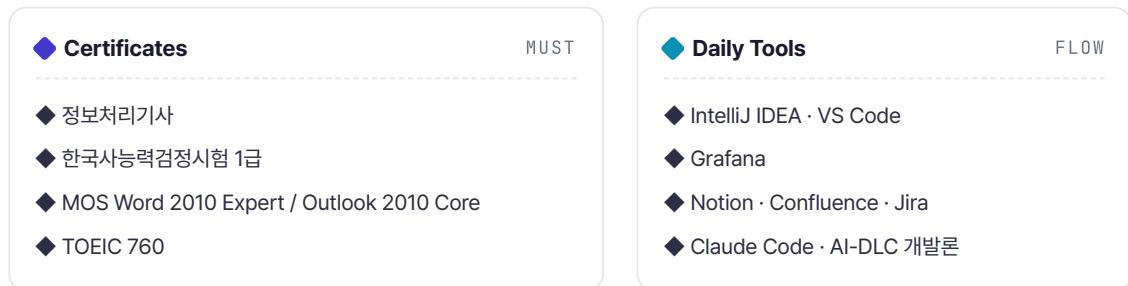
개인적으로는 **AI-DLC 개발론**을 적극 도입해 Claude Code · Cursor · Codex 등 AI 에이전트와 협업하며, **코인 자동매매 · AI 분석 알림** 사이드 프로젝트를 풀스택으로 운영 (Telegram 구독자 830+)하고 있습니다.



카테고리별로 분류한 기술 스택입니다. **Backend / Architecture**가 주력 영역이며, 운영 환경 전반에 걸친 **End-to-End 책임**을 가지고 일하는 것을 선호합니다.



CHAPTER 03 · b Certificates & Tools



DOMAIN LEAD · 2022.07 ~

베네피아 — 주문/결제 도메인 담당

3,700개 고객사 임직원이 사용하는 복지포인트 쇼핑몰의

주문 · 결제 · 배송 시스템 운영 및 개발

Java

Spring Boot

MyBatis

Oracle

RabbitMQ

Docker

Nomad / Consul

JSP

담당 역할 — 주문 · 결제 · 배송 도메인 담당으로서 운영 모니터링 · 장애 대응 · 데이터 추출 등을 도맡았으며, 프론트 및 백엔드 풀스택을 담당하였습니다.

1 MSA에서의 주문 · 배송 · 결제 전면 재설계

2024.12 — 2025.06

◆ WHY

- 상품·배송비·주문이 강하게 결합된 구조
- 정책 변경에 유연한 대응 불가

◆ HOW

- 주문에서 상품·배송비를 **N:N으로 분리**
- 취소/반품/교환 프로세스 추가
- **역등성 · 동시성 · 분산 트랜잭션** 제어
- 객체지향적 리팩토링
- 실시간 모니터링을 통한 운영 개선
— 재고 동시성 처리, 포인트 연마감 결제취소 이슈 등 주요 오류 **근본 원인 분석·개선**

◆ IMPACT

- 다양한 정책 변경에 유연 대응
- 조건부 배송비 등 복잡 케이스 처리
- 유지보수 비용 절감
- 초기 대비 **장애율 76.5% 감소**

2 주문 핵심 시나리오 중심 테스트 코드 도입

73%↓ MTTR

0건 운영 장애

◆ WHY

- 정책 변경 시 기존 기능 장애 우려
- 사이드이펙트 확인으로 개발 속도 저하

◆ HOW

- MSA 통합 테스트 환경 구축
- 주문·결제·배송 핵심 시나리오 테스트
- Persistence Layer 테스트환경 구축

◆ IMPACT

- MTTR **45분 → 12분 (73% 단축)**
- 운영 장애 0건 유지

◆ CHALLENGE

- **persistence layer 환경구축 문제** → 테스트 이후의 개발 DB 오염, 분산 트랜잭션 롤백 필요
- MSA에서의 DB 공유로 인한 타 API mocking 실패
- **선행 데이터 구축** → 주문테스트 진행을 위해선 **상품데이터**가 필요

3 현금영수증 고도화

RabbitMQ

① MQ 도입을 통한 주문 처리 구조 개선, ② 벤더별 현금영수증 발급 구조 개선, ③ 현금영수증 운영 어드민 기능화, ④ AI-DLC 개발론으로 AI 주도개발 도입

◆ WHY

- 외부 플랫폼 통신 지연 → 취소 누락
- 후행 작업 실패가 주문 실패로 전파
- 베네피아 명의 일괄 발급 → 실제 매출 주체 미반영 · 발급/취소 누락 수 기 처리

◆ HOW

- 주문 완료 이벤트 기준 MQ 분리
- 거래유형별 **벤더사 사업자번호** 발급 · 현금영수증 **어드민 기능화**
- AI-DLC 개발론**으로 AI기반 개발

◆ IMPACT

- 장애 전파 차단
- 확장성 / 가용성 확보
- 대규모 트래픽 대응
- 실제 매출 귀속 주체 발급 · 운영 수 기 처리 감소
- AI-DLC 개발론으로 개발하여 **공수 (MM) 66% 절감**

◆ CHALLENGE

- 메시지 유실 → 인프라 및 각 아키텍처에 대한 **Life Cycle**을 이용
- 순서제어 불가 → 샤딩 or **Single Consumer** 도입
- 분산 **Consumer** 환경의 **race condition** · **Lost Update** → 낙관적 락 도입
- 오류 제어 → 재시도 큐와 **DLQ**를 적용해 오류제어

4 AI Tech TF — AI 도입 리드

2026.05 — 재직중

◆ WHY

- 정책·코드·운영 데이터가 분리되어 AI가 **문맥 연결 불가**
- 보안상 **운영 DB 직접 접근 불가** → LLM 업무 처리 범위 축소, 부정확한 개발
- AI 산출물의 코드 컨벤션 미통일
- 아키텍처·정책·비용·보안 구조 미인 지 → **AI 환각 발생**, 컨텍스트·토큰·응답시간 낭비

◆ HOW

- AI-DLC** 기반 도입 전략 주도
- AI 아키텍처** 설계
- Claude Code Harness Engineering**

◆ IMPACT

- AI 아키텍처 설계**로 외부 LLM 활용 시에도 운영 데이터 유출 차단 구조 확보
- Harness Engineering** 기반으로 정책·코드·운영 데이터를 AI가 한 번에 연결해 운영 이슈 분석 속도 향상
- AI-Driven Development Lifecycle** 관점 개발을 통해 프로젝트 개발 공수 MM 대폭 감소
- Claude Code** 환경 구축으로 AI 산출물 품질 향상·통일화
- Harness Engineering** 기반 AI 환각 차단 + 컨텍스트·토큰·응답시간 효율화

STORY 01 · MESSAGING

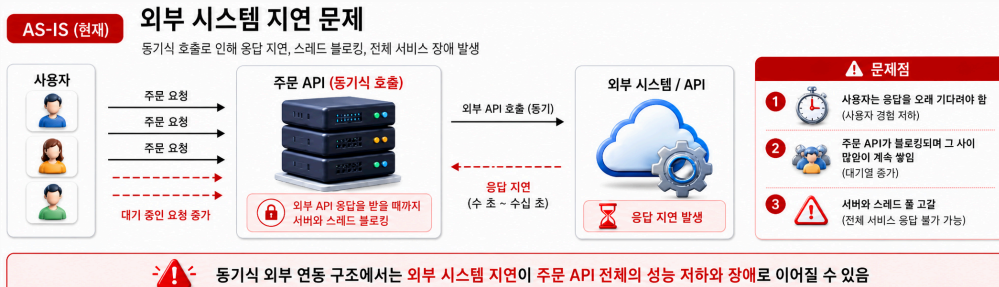
RabbitMQ 도입 — 동기식의 한계 ① · ②

WHY?

초기에는 주문과 관련된 모든 처리를 동기식으로 구현했는데, 실제 운영을 하다 보니 몇 가지 한계가 명확히 드러났습니다.

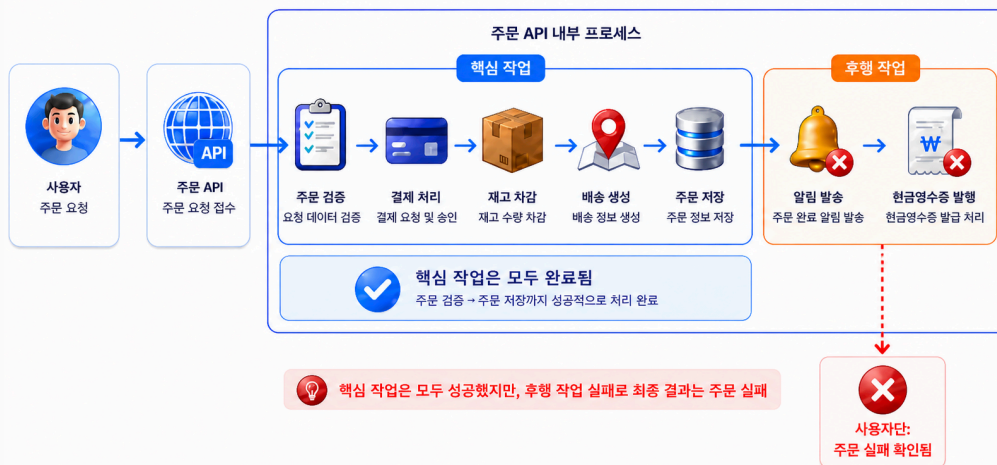
① **외부 시스템 지연** — 동기식으로 외부 API를 호출하는데 응답이 1분 이상 걸리는 경우가 있어 사용자가 그동안 기다려야 했고, 주문 API가 그 시간 동안 블로킹되며 들어오는 요청들이 쌓여 **서블릿 스레드 풀이 고갈**되어 전체 서비스가 응답하지 못할 수 있었습니다.

AS-IS · 외부 시스템 지연 문제



② **후행 작업의 장애 전파** — 결제 완료 후 현금영수증 API 호출까지 하나의 주문 프로세스로 묶여 있었습니다. 발급이 실패하면 주문 자체가 롤백되는, 비즈니스 중요도에 비해 과도한 결합 구조였습니다. try-catch로 잡아도 OOM 같은 장애에는 같은 서버이기 때문에 동반 다운에 취약합니다.

AS-IS · 후행 작업의 장애 전파



STORY 01 · MESSAGING (continued)

RabbitMQ 도입 — 동기식의 한계 ㉠·㉡·솔루션

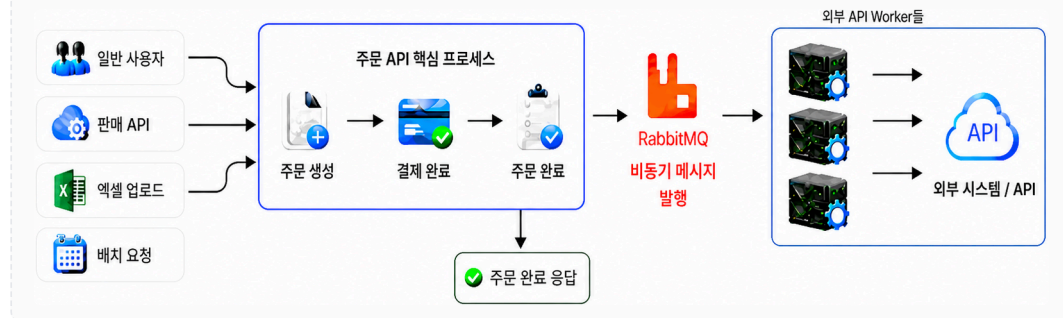
WHY? · 계속

㉠ **대용량 트래픽 대응** — 일반 사용자는 모바일/PC에서 한 건씩 주문하지만 벤더사는 API 호출이나 엑셀 업로드로 대량의 주문을 동시에 호출할 수 있습니다. 이런 요청이 동시에 몰릴 때 주문 API가 다운될 위험이 있었습니다.

㉡ **대량 INSERT 배치성 작업** — 단일 트랜잭션으로 처리하다 보니 타임아웃, 롤백 비용, DB 부하가 반복적으로 발생했습니다. MQ를 도입하면 Queue가 **버퍼 역할**을 하여 대규모 트래픽에도 장애 없는 시스템을 구축할 수 있습니다.

RabbitMQ를 통해 긴 지연시간은 **비동기 호출**로 해결하고, 분리된 환경이기 때문에 장애 전파와 OOM도 방지할 수 있습니다. **대규모 트래픽 케이스에서도 병목 지점을 파악하여 MQ를 도입해 안정성 있게 설계**할 수 있었습니다.

TO-BE · RABBITMQ 비동기 메시징 구조



STORY 01 · MESSAGING (continued)

RabbitMQ 도입 — 진행 중 마주친 3가지 문제

CHALLENGE 01 · 메세지 유실

DB 락 때문에 재기동을 해야 하거나 배포로 scale-out된 서버들을 재기동해야 할 때, 진행 중이던 메시지가 **유실될 수 있는 위험**이 있었습니다.

- ◆ MQ 미들웨어 고장
- ◆ Docker, Nomad, Consul 등 인프라 Shutdown 연계 및 LifeCycle 확인 필요

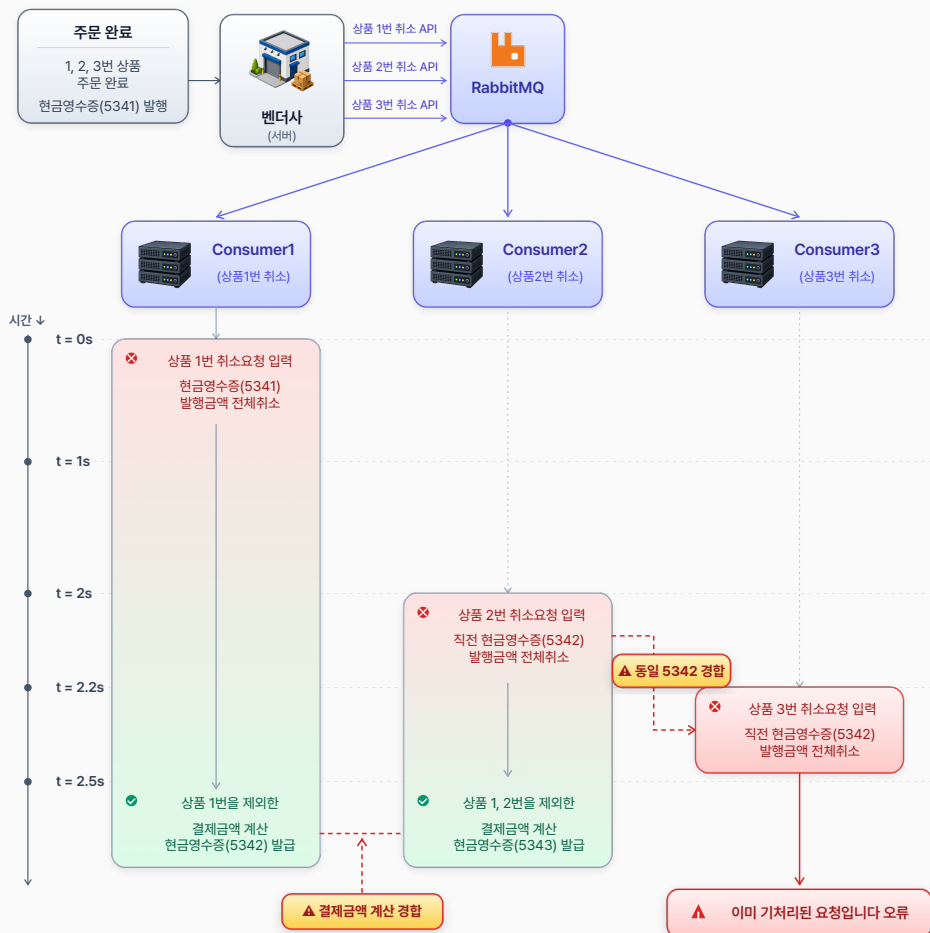
STORY 01 · MESSAGING (continued)

RabbitMQ 도입 — 진행 중 마주친 3가지 문제

CHALLENGE 02 · 순서 제어 및 동시성 제어

주문 부분취소 시 국세청에는 부분취소가 없기 때문에, 이전 현금영수증 처리 금액을 **전액 취소 API**로 요청한 뒤 남은 결제 금액을 다시 호출해야 합니다. 아래처럼 **동시다발적으로 취소요청 API 호출**하는 경우 문제가 생길 수 있습니다.

PROBLEM · 부분 취소 동시성 시나리오 (세로 흐름)



시도한 방법들

- ◆ 샤딩 — $\text{hash}(\text{ordNo}) \% 4$ 로 큐 4개 분산. 같은 ordNo는 항상 같은 큐 → 같은 서버
- ◆ x-single-active-consumer — 큐 단위 단일 Consumer 보장
- ◆ Exclusive Consumer로 순서제어 ← 1차 채택 ✓
- ◆ 분산 Consumer 환경 + 동시성제어 ← 최종 채택 ✓

STORY 02 · QUALITY

통합 테스트 환경 구축 — Testcontainers 기반 격리 환경

PROBLEM

주문 도메인은 **배송 · 결제 · 쿠폰 · 포인트 · 상품** 여러 도메인이 정책으로 얹혀 있고, 결제 · 취소 · 반품 등 시나리오 또한 다양합니다. 일부 기능 수정·정책 추가만으로도 **사이드 이펙트와 광범위한 수동 검증**이 따라 개발 효율을 크게 떨어뜨렸기에, **핵심 usecase 대상 시나리오 테스트 코드 도입**을 진행했습니다.

난관 1 · PERSISTENCE 전략 선택

통합 테스트 수행 시 **persistence layer(데이터 계층)**를 어떻게 다룰지가 핵심 고민이었습니다. Mock 기반 테스트 · Alpha DB · 테스트 전용 DB · In-Memory DB · Testcontainers 등 여러 대안을 검토하며, MSA 환경에 적합한 격리 전략을 선택해야 했습니다.

난관 2 · 개발 DB 오염

초기에는 테스트 환경에서도 개발 DB에 붙여 통합 테스트를 진행했습니다. 구성은 쉬웠지만 **외부 트래픽으로 인한 환경 오염**이 문제였습니다. 예를 들어 "주문 1건 → 주문 수 1 증가" 검증 시, 동시간대 외부에서 발생한 주문이 카운트에 섞이며 검증이 깨졌습니다. **범위 · 정산 · 통계성 테스트에 부적합**한 구조였습니다.

난관 3 · 분산 트랜잭션 롤백

MSA 특성상 **서비스 간 트랜잭션 경계가 분리**되어 있어, 주문 API 테스트 중 포인트 API를 호출해 포인트가 발급되면 테스트 종료 시 해당 포인트를 별도로 롤백해야 했습니다. **분산 트랜잭션 환경에서의 데이터 정리**가 테스트 안정성을 크게 저해했습니다.

ORDER API · 호출 흐름



STORY 02 · QUALITY (continued)

통합 테스트 환경 구축 — Testcontainers 기반 격리 환경

난관 4 · 타 API 모킹

MSA 내 타 프로젝트와 통신할 때의 모킹 문제도 있었습니다. 주문 API가 포인트 API를 호출하는데, 포인트 API 측에 주문 API의 DB에 쓰는 로직이 섞여 있어 **주문 API 테스트가 포인트 API 구현에 의존**하게 되었습니다. DB를 공유한 설계 자체가 잘못이었음을 깨닫고, **MSA의 경계 처리**가 테스트 가능성과 직결됨을 확인했습니다.

MSA 경계 · 의존성 도식



DECISION

- ◆ Testcontainers로 격리 환경 구축 — 컨테이너 재기동만으로 분산 트랜잭션 롤백을 한 번에 해소
- ◆ 포인트 API의 **주문 도메인 로직**을 **주문 API로 재배치**해 서비스 경계 명확화
- ◆ 경계 정리 후 포인트 API는 **mocking으로 대체**, 테스트 대상 영역만 명확히 검증

OUTCOME

외부 환경에 영향받지 않는 **격리된 테스트 환경**을 확보하고, 실제 API 통신 없이 안정적으로 테스트할 수 있게 되었습니다. 테스트의 **재현성과 신뢰성**이 크게 향상되었으며, MSA에서의 **경계 설계 원칙**을 실제 코드 구조에 반영하는 계기가 되었습니다.

STORY 03 · SECURITY

MSA에서의 XSS 처리 대응 — 전사 공통 기준 수립

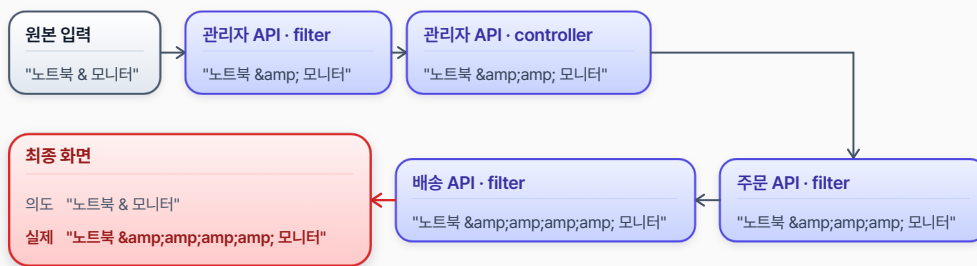
PROBLEM

MSA 환경에서 서비스별로 상이하게 적용되던 XSS 대응 방식으로 발생한 **이중 이스케이프 · 데이터 변형 · JSON 파싱 오류**를 해결했습니다.

CONSTRAINT

각 서비스마다 필터 · 인터셉터 · 화면단 등 서로 다른 위치에서 XSS 처리를 수행하면서, 동일 데이터가 여러 API를 거치는 과정에서 중복 이스케이프가 발생했습니다. 데이터 변형 및 비정상 노출이 발생하였고, MSA 구조 특성상 여러 서비스를 동시에 고려해야 해 **영향 범위가 넓고 검증이 어려운 제약**이 있었습니다.

XSS 다중 이스케이프 — 주문 조회 흐름



DECISION

개별 이슈 대응이 아닌 **전사 공통 XSS 처리 기준** 수립으로 접근했습니다. 정보보안 부서와 협업해 정책을 정의하고, **애노테이션 기반 공통 처리 구조**를 설계해 전사 가이드로 정리·배포하고 적용을 주도했습니다. 기존 데이터 흐름을 분석해 중복 처리 및 데이터 변형 발생 구간을 식별하고, 서비스 전반에 걸쳐 일관된 방식으로 정비했습니다.

OUTCOME

서비스 간 XSS 처리 방식이 일관되게 정리되어 데이터 변형 및 이중 처리 문제가 해소되었습니다. 전사 가이드 적용으로 **조직 전반의 보안 수준이 향상**되었으며, ISMS 점검에서 **XSS 관련 지적 건수가 크게 감소**하는 성과를 달성했습니다.

STORY 04 · DESIGN

레거시 개선 — 결제 도메인의 전략 패턴 적용

PROBLEM

주문 결제 도메인에서 다양한 결제 수단(카드 · 포인트 · 간편결제 등)과 복합 결제를 처리하면서 발생하던 **if-else 기반 분기 로직 증가** 및 변경 시 사이드 이펙트 문제를 해결했습니다.

CONSTRAINT

결제 도메인은 정책 변경과 신규 결제 수단 추가가 빈번한 영역으로, 기존 구조에서는 기능이 추가될 때마다 조건문이 계속 증가하고 기존 코드를 수정해야 했습니다. 변경 범위가 넓어지고 작은 수정에도 예상치 못한 오류가 발생할 수 있는 구조로, **유지보수성과 확장성이 크게 저하**된 상태였습니다.

DECISION

조건문을 줄이는 것이 아니라 **변경의 책임을 객체로 분리**하는 방향으로 접근했습니다.

- ◆ 결제 방식을 객체로 추상화 후 **전략 패턴(Strategy Pattern)** 도입
- ◆ 각 결제 수단이 자신의 행위를 스스로 처리하도록 설계
- ◆ 클라이언트 코드가 구체 구현이 아닌 **추상화에 의존**하도록 개선

BEFORE · 절차적 분기

```
if (m == CARD)      { ... }
else if (m == POINT) { ... }
else if (m == COMPOSITE) {
    // 카드 + 포인트 분기 또 분기...
} else if ...
```

AFTER · 전략 패턴

```
interface PaymentStrategy {
    Result pay(Order o);
}

class CardPay implements PaymentStrategy { ... }
class PointPay implements PaymentStrategy { ... }
class CompositePay implements PaymentStrategy { ... }
```

SPRING · MAP 기반 BEAN 자동 주입

```
@Component("CARD")      class CardPay implements PaymentStrategy { ... }
@Component("POINT")      class PointPay implements PaymentStrategy { ... }
@Component("COMPOSITE")  class CompositePay implements PaymentStrategy { ... }

@Service
@RequiredArgsConstructor
class PaymentService {
    private final Map<String, PaymentStrategy> strategies; // ← Spring이 Bean 이름 → 인스턴스로 자동 주입
}
```

OUTCOME

조건문 중심의 절차적 구조에서 벗어나 **다형성 기반의 객체지향 구조**로 개선되어, 결제 로직의 확장성과 유지보수성이 크게 향상되었습니다. 정책 변경이나 신규 결제 수단 추가 시 **기존 코드를 수정하지 않고 확장**할 수 있는 OCP 만족 구조를 확보했습니다.

PERSONAL · 830+ Subscribers · AI-Native

실시간 코인 자동매매 & AI 분석 알림 서비스

코인 시장의 실시간 변동성을 분석하여 **지표 개발** → **백테스트** → **자동매매** → **알림 서비스화** 전 과정을 1인 개발로 운영 중인 사이드 프로젝트입니다. 설계 단계부터 **AI-DLC 개발론**을 적극 도입해 Claude Code · Cursor · Codex 등 AI 에이전트와 협업하며 개발 속도를 가속화했고, 실 서비스에는 **OpenAI · Claude API**를 연동하여 시장 분석 리포트 자동화와 사용자 맞춤 시그널 해석 기능을 제공합니다. React 기반 수익 포트폴리오 대시보드까지 직접 구축하며 **풀스택 + AI 통합 역량**을 실전에서 검증한 프로젝트입니다.

Node.js React OpenAI API Claude API

AI-DLC 개발론 Binance API Upbit API

TradingView Script Multi-process



AI-DLC 개발론

AI 에이전트와 함께 짓는 코드

Claude Code · Cursor · Codex CLI 등 AI 에이전트를 활용한 **AI-DLC 개발론**을 도입했습니다. 사양 명세 → 테스트 우선 작성 → 구현 → 셀프 코드 리뷰까지 반복 작업을 AI에 위임하고, 사람은 아키텍처 설계와 의사결정에 집중하는 구조로 운영합니다. 그 결과 1인 개발 환경에서도 백엔드 · 프론트엔드 · 데이터 분석을 동시에 진행할 수 있었습니다.



AI SERVICE INTEGRATION

AI를 서비스에 통합

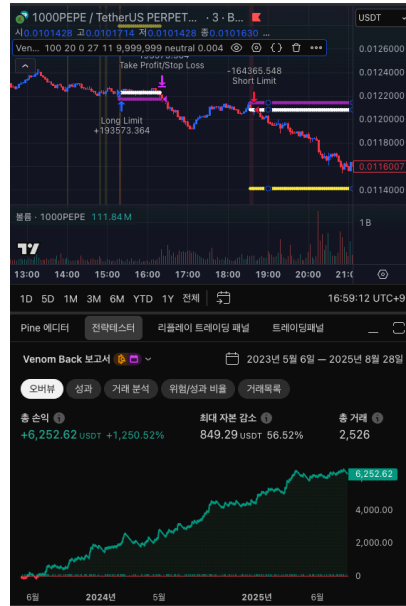
OpenAI · Claude API를 결합하여 실시간 시장 데이터를 요약 · 해석하는 AI 분석 파이프라인을 구축했습니다. Function Calling으로 시장 지표를 도구화하고, **Prompt Caching**으로 반복 호출 비용을 절감했습니다. 최종적으로 급등락 알림에 **AI 코멘트**를 자동 첨부하여 "왜 지금 시그널이 발생했는가"를 사용자가 바로 이해할 수 있게 만들었습니다.



01 · INDICATOR

지표 개발

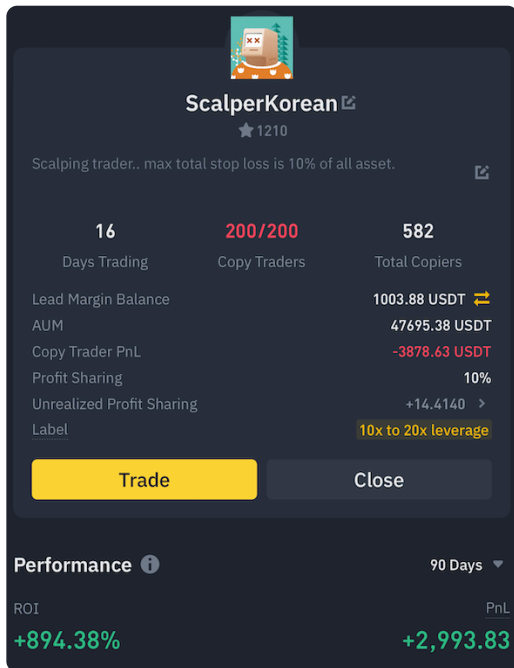
거래량 폭등 · 추세 반전 캔들패턴 기반의 **역추세 전략 지표**를 직접 설계 · 구현. **TradingView Pine Script**로 시각화하고 조건 충족 시 웹훅 자동 알림을 발송하도록 구성했습니다. 지표 파라미터 튜닝 단계에서는 AI 에이전트와 페어 프로그래밍을 통해 빠르게 가설을 검증했습니다.



02 · BACKTEST

전략 검증

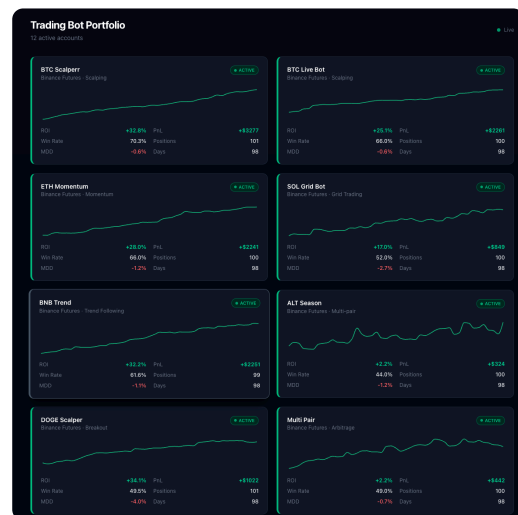
수백만 개 파라미터 조합을 **Node.js 멀티프로세스 병렬**로 백테스트 하여 단일 프로세스 대비 약 **10배 향상**의 처리량을 확보했습니다. MDD · Volatility · PSI · Sharpe 등 다중 지표로 평가하며, 시드값 고정으로 **재현성 99%**를 달성했습니다.



03 · TRADING

자동매매 시스템

검증된 전략을 **Binance Futures API** 기반 자동매매 봇으로 구현. 프로세스 이중화 · WebSocket Heartbeat · 자동 재기동 메커니즘을 통해 **24/7 무중단 가용성**을 확보하고, 슬리피지 · API rate limit 까지 직접 핸들링했습니다.



04 · AI ALERT

AI 분석 알림 & 대시보드

실시간 모니터링으로 급등 · 급락 발생 시 Telegram · Discord로 자동 알림 전송. **LLM이 시장 컨텍스트를 분석한 코멘트**를 첨부하고, React 기반 수익 포트폴리오 대시보드에서 전략 성과를 한눈에 확인할 수 있도록 했습니다. **Telegram 구독자 830명** (2025.12 기준) 규모로 운영 중.

OPEN TO OPPORTUNITIES

신뢰성 높은 시스템을
함께 만들어 보고 싶습니다.

emrhssla@gmail.com[GitHub ↗](#)[Tech Blog ↗](#)[Notion ↗](#)[Tistory ↗](#)